Fundamentals of Artificial Intelligence

Adversarial Search
Game Playing

Games

- In **multiagent environments**, each agent needs to consider the actions of other agents and how they affect its own welfare.
- The agents can be **cooperative** or **competitive**.
- In **competitive environments**, the agents' goals are in conflict, giving rise to **Adversarial Search problems** — often known as **GAMES**.
- In mathematical **game theory**, a multiagent environment is treated as a **game**, the impact of each agent (economy) on the others is significant, regardless of whether the agents are ***cooperative*** or ***competitive***.
- In **AI**, the most common games are *deterministic*, *turn-taking*, *two-player*, **zero-sum games** of **perfect information** (such as chess).
  - In deterministic and fully observable environments in two agents act alternately and in which the utility values at the end of the game are always equal and opposite.
  - If one player wins a game of chess, the other player necessarily loses.
  - It is this opposition between the agents' **utility functions** that makes the situation **adversarial**.

Games

- Games are too hard to solve.
  - Chess has an average branching factor of about 35, and games often go to 50 moves by each player, so the search tree has about 35^{100} nodes.
- Games require the ***ability to make decision even when calculating optimal decision is infeasible***.
- **Pruning** allows us to ignore portions of the search tree.
- **Heuristic evaluation functions** allow us to approximate **true utility of a state** without doing a complete search.
- Games such as backgammon includes elements of **imperfect information** because not all cards are visible to each player.
- Types of games

	deterministic	**chance**
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleships,	bridge, poker, scrabble

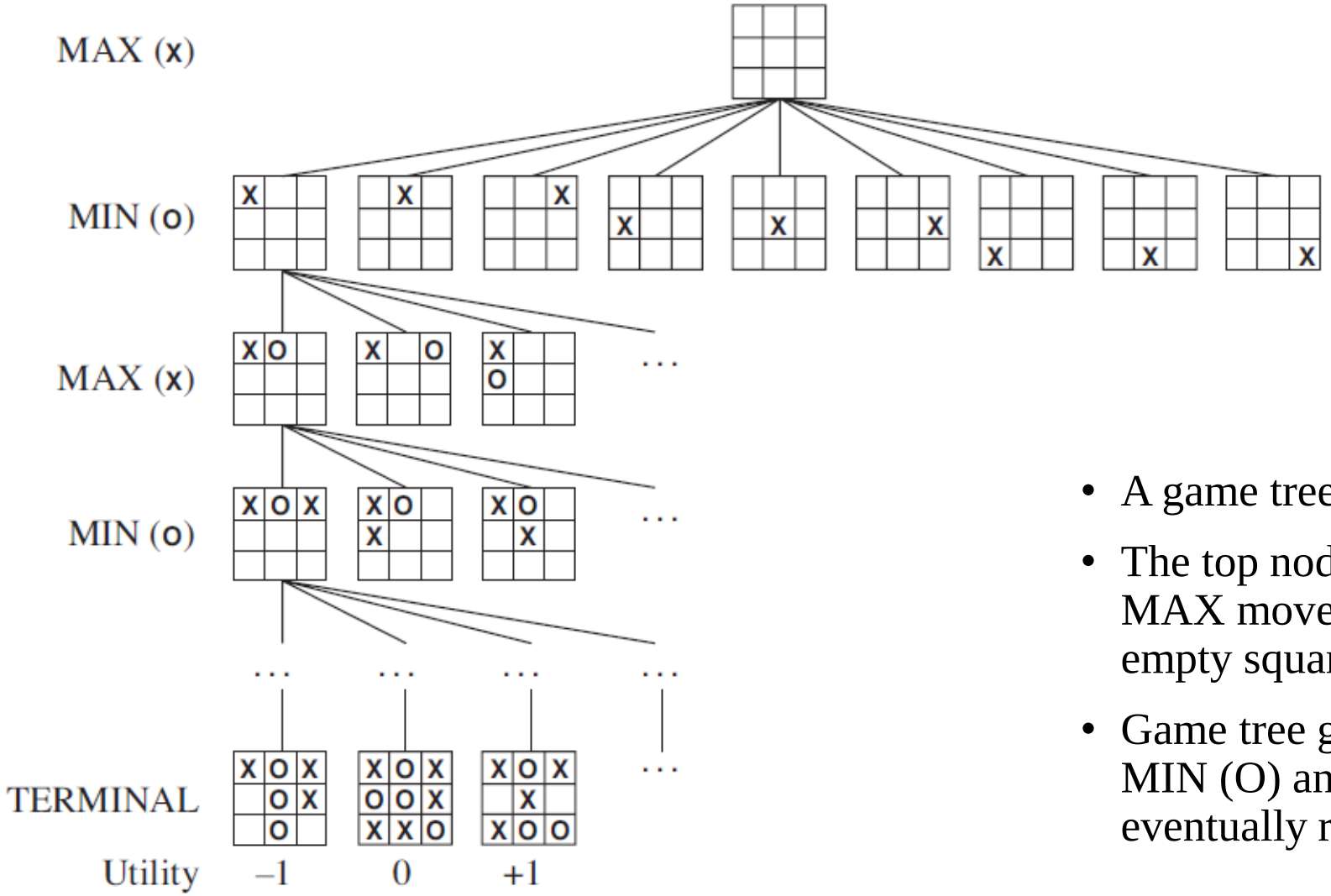
Games as Search Problem

- S_0: The **initial state**, which specifies how the game is set up at the start.
- PLAYER(s): Defines which player has the move in a state.
- ACTIONS(s): Returns the set of legal moves in a state.
- RESULT(s, a): The **transition model**, which defines the result of a move.
- TERMINAL-TEST(s): A **terminal test**, which is true when the game is over and false otherwise. States where the game has ended are called **terminal states**.
- UTILITY(s, p): A **utility function** (also called **objective function** or **payoff function**), defines the final numeric value for a game that ends in terminal state s for a player p.
  - In chess, the outcome is a win, loss, or draw, with values +1, 0, or 1/2 .
  - A **zero-sum game** is (constant-sum is a better term) defined as one where the total payoff to all players is the same for every instance of the game.
  - Chess is zero-sum because every game has payoff of either 0+1, 1+0 or 1/2+1/2.

Games as Search Problem

- The initial state, ACTIONS function, and RESULT function define the **game tree** for the game—a tree where the **nodes are game states** and the **edges are moves**.
  - For tic-tac-toe the game tree is relatively small—fewer than 9! = 362, 880 terminal nodes.
  - But for chess there are over 10^{40} nodes, so the game tree is best thought of as a theoretical construct that we cannot realize in the physical world.
- We have two players: MAX and MIN.
  - MAX (our player) moves first in the game.
- Regardless of size of game tree, MAX's (our player's) job to search for a good move in search tree.
  - A search algorithm does not explore the full game tree, and examines enough nodes to allow a player to determine what move to make.

Game Tree



- A game tree for the game of tic-tac-toe.
- The top node is the initial state, and MAX moves first, placing an X in an empty square.
- Game tree gives alternating moves by MIN (O) and MAX (X), until eventually reaching terminal states.

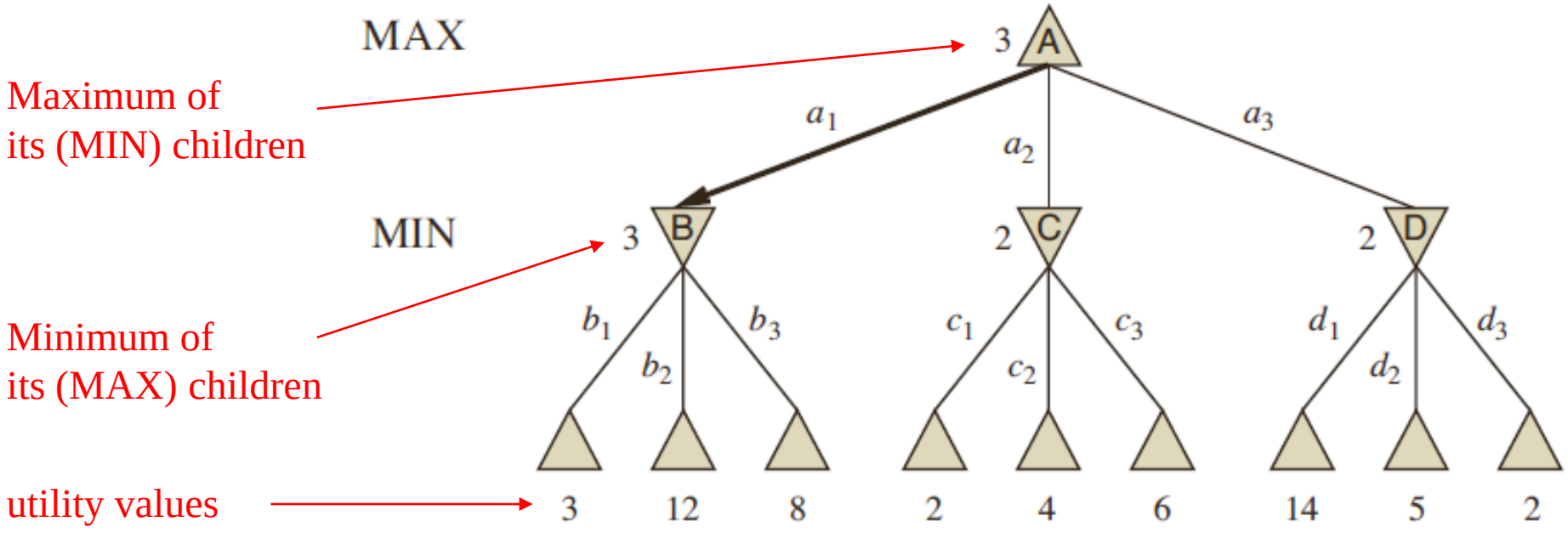
Optimal Decisions in Games

- **Optimal Solution** in a search problem is a sequence of actions leading to a ***goal state*** (***a terminal state that is a win).***
- MAX must find a contingent **strategy**, which specifies MAX's move in the initial state, then MAX's moves in the states resulting from every possible response by MIN.
- Given a game tree, **optimal strategy** can be determined from **minimax value** of each node n, and it is called as MINIMAX(n).
- **The minimax value of a node** ***is the utility (for MAX) of being in the corresponding state, assuming that both players play optimally from there to the end of the game***.
- The **minimax value of a terminal state** is just its ***utility***.
- MAX prefers to move to a state of maximum value, whereas MIN prefers a state of minimum value.

$$\textsc{Minimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } \textsc{Terminal-Test}(s) \\ \max_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{max} \\ \min_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{min} \end{cases}$$

MINIMAX

- Δ nodes are "MAX nodes," in which it is MAX's turn to move, and ∇ nodes are "MIN nodes."
- Terminal nodes show the utility values for MAX; other nodes are labeled with their minimax values.
- MAX's best move at the root is a1, because it leads to the state with the highest minimax value, and MIN's best reply is b1, because it leads to the state with the lowest minimax value.



Minimax Algorithm

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v
```

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

- Minimax Algorithm returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility.

- The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state.

Properties of Minimax Algorithm

- The minimax algorithm performs a complete depth-first exploration of the game tree.

Complete: **YES**, if tree is finite

Optimal: **YES**, against an optimal opponent.

Time complexity: **$O(b^m)$**

Space complexity: **$O(bm)$** (depth-first exploration) Algorithm generates all actions at once

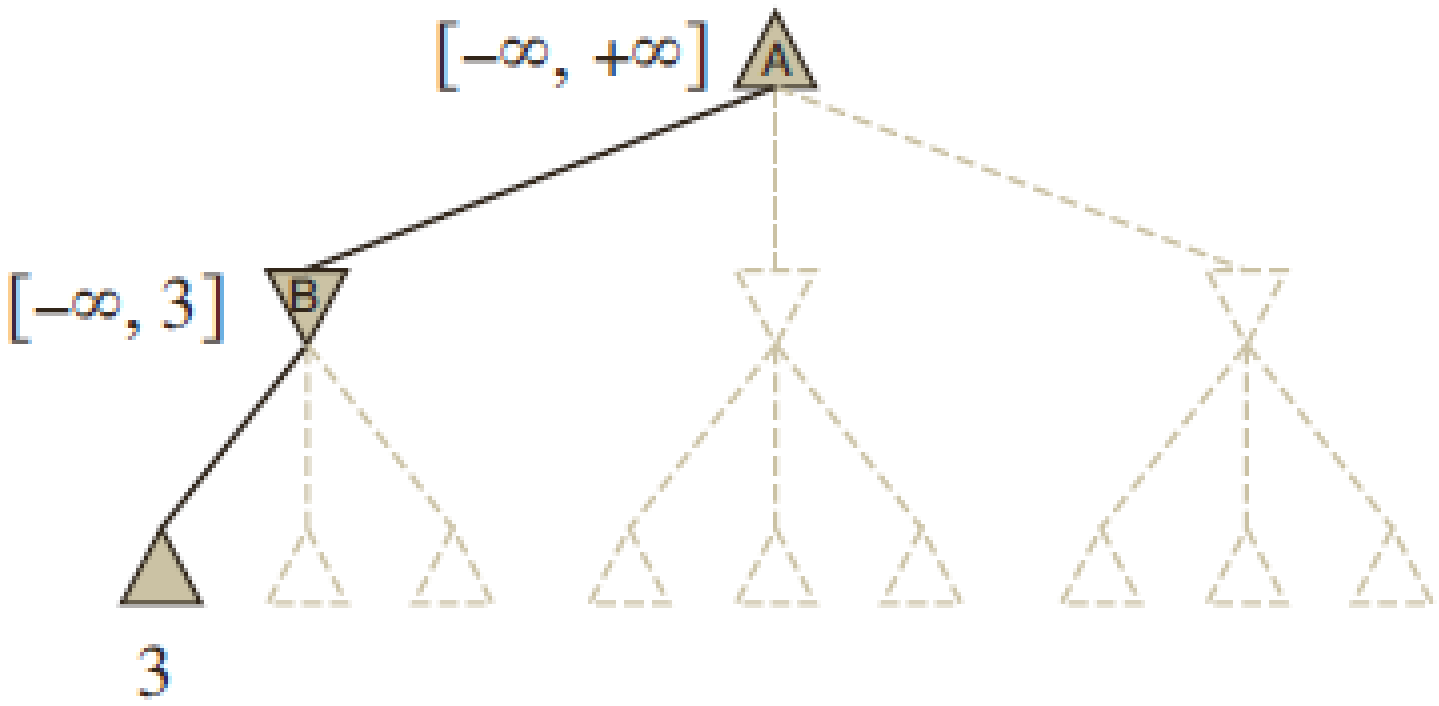
- For real games, the time cost is totally impractical, but minimax algorithm serves as the basis for the mathematical analysis of games and for more practical algorithms.
- For chess, $b \approx 35$, $m \approx 100$ for reasonable games ➔ exact solution completely infeasible

Alpha–Beta Pruning

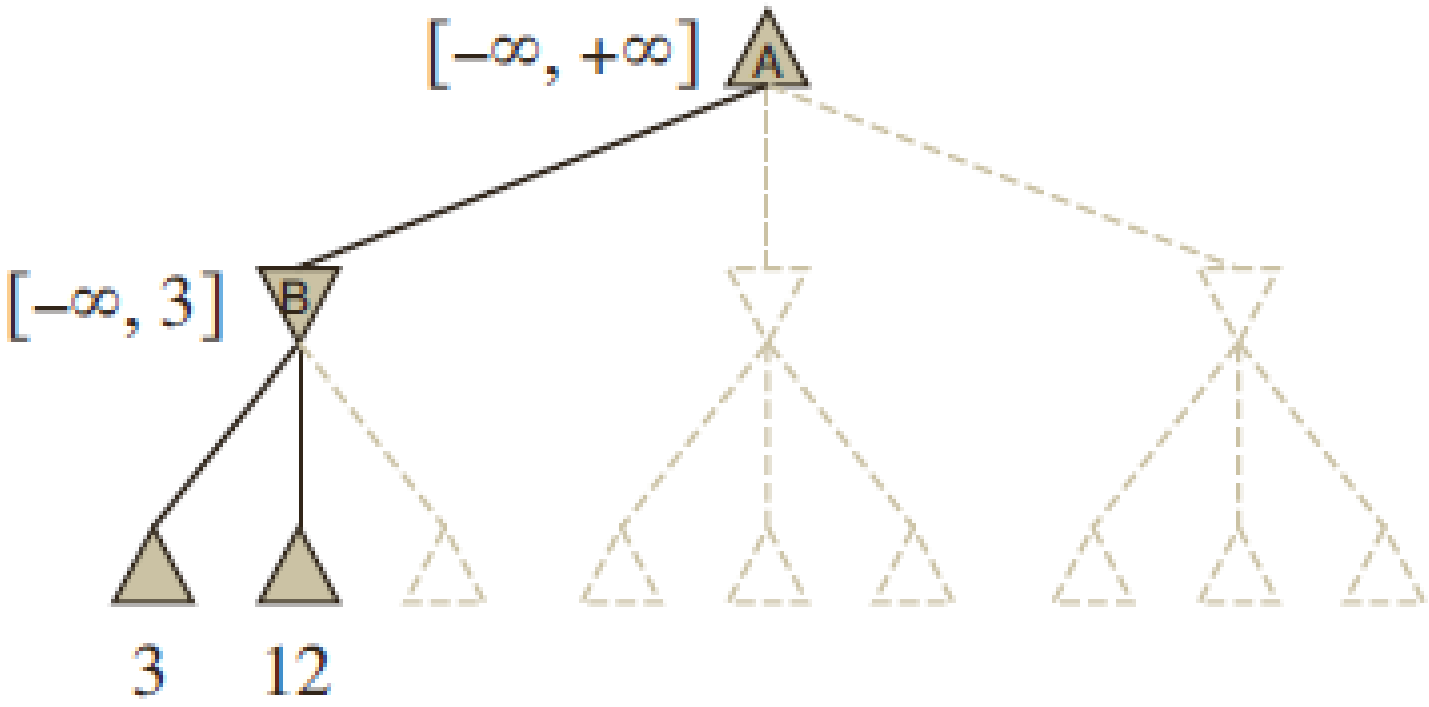
- The problem with minimax search is that the number of game states it has to examine is exponential in the depth of the tree.
- Unfortunately, we can’t eliminate the exponent, but we can effectively cut it in half.
- It is possible to compute **correct minimax decision** without looking at every node in game tree.
- **Alpha–Beta pruning** prunes away branches of the minimax tree that cannot possibly influence the final decision and it returns the same move as minimax algorithm would.

- ***α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.***
- ***β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.***

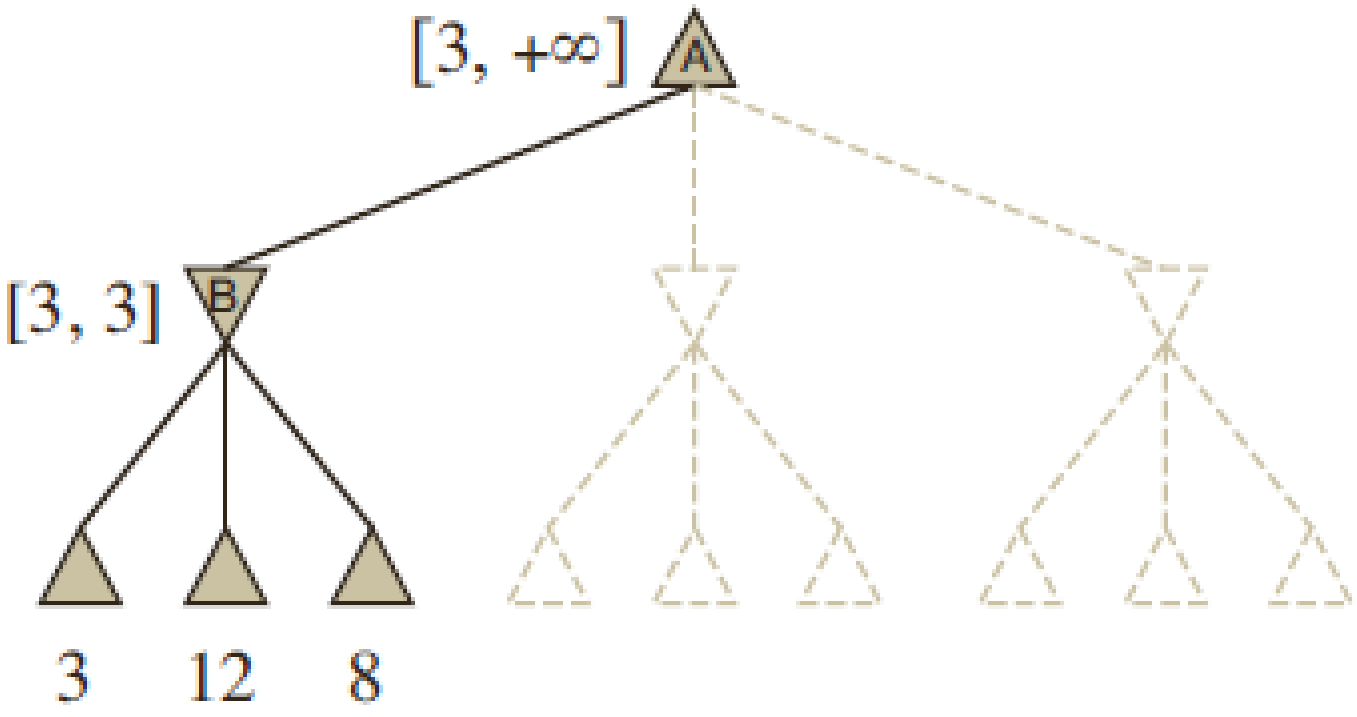
Alpha–Beta Pruning Example



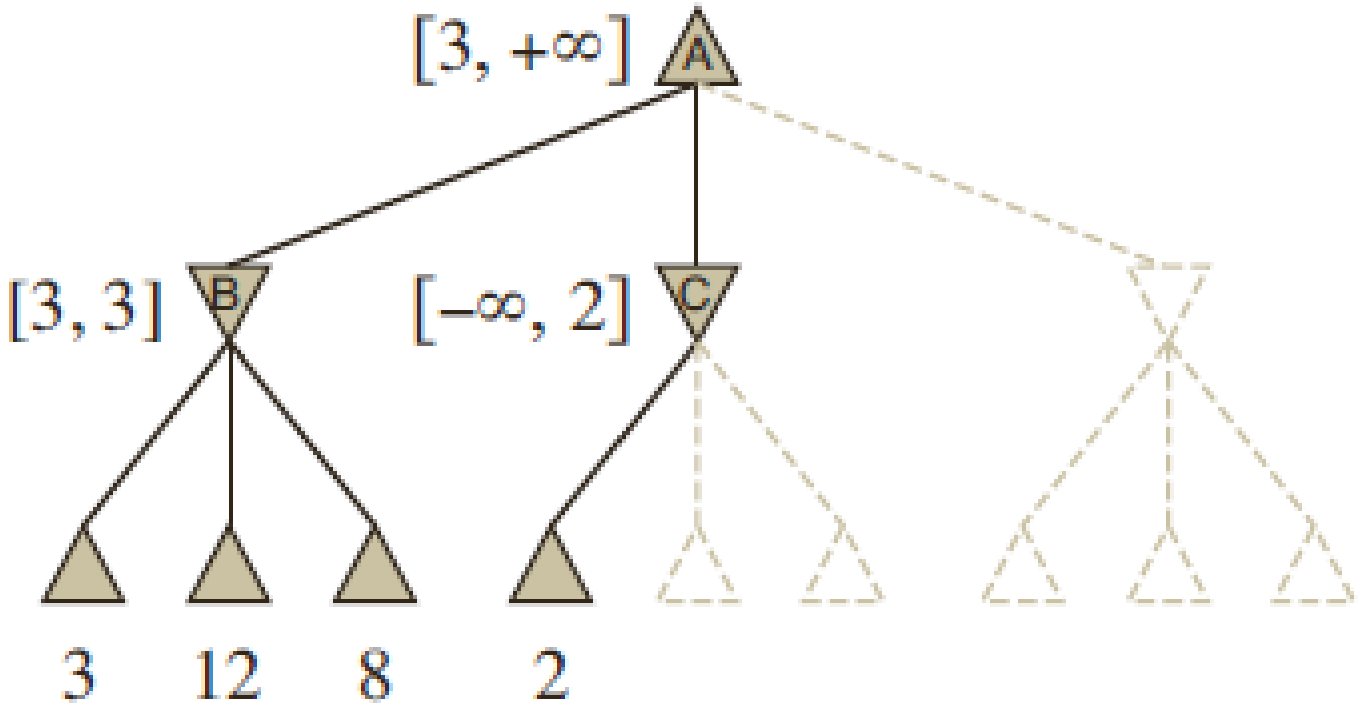
Alpha–Beta Pruning Example



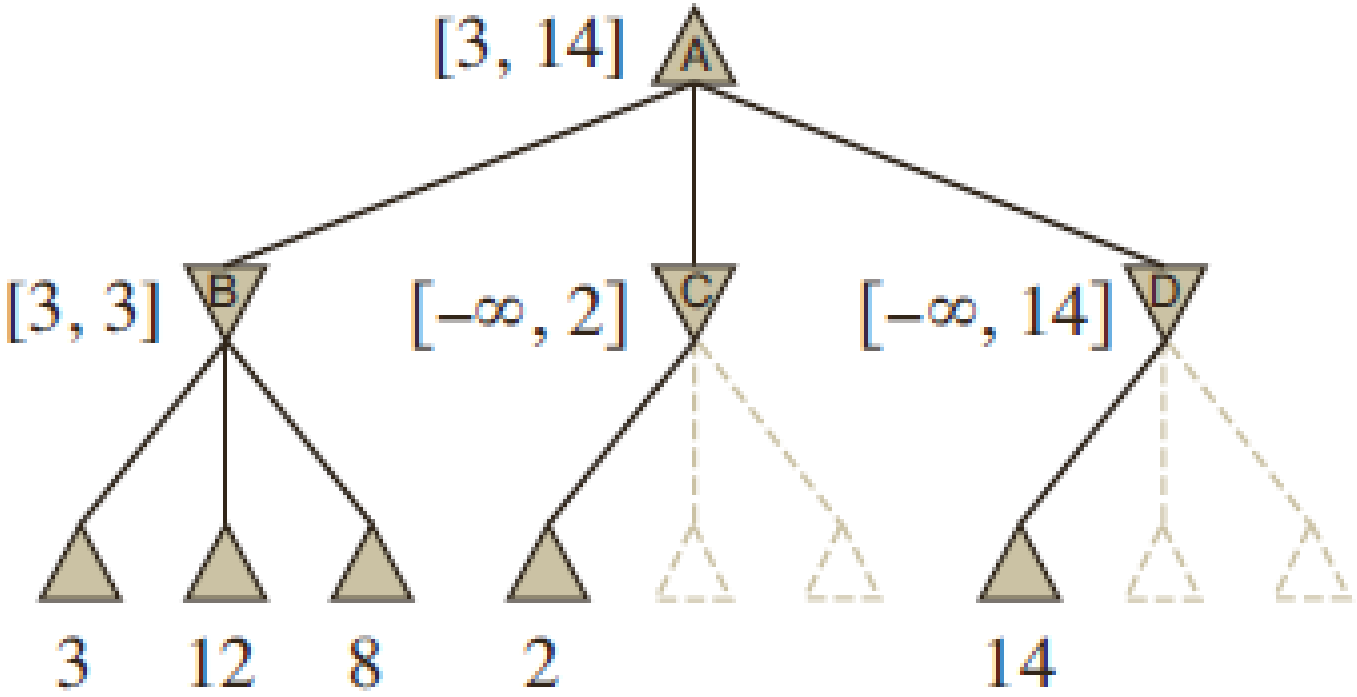
Alpha–Beta Pruning Example



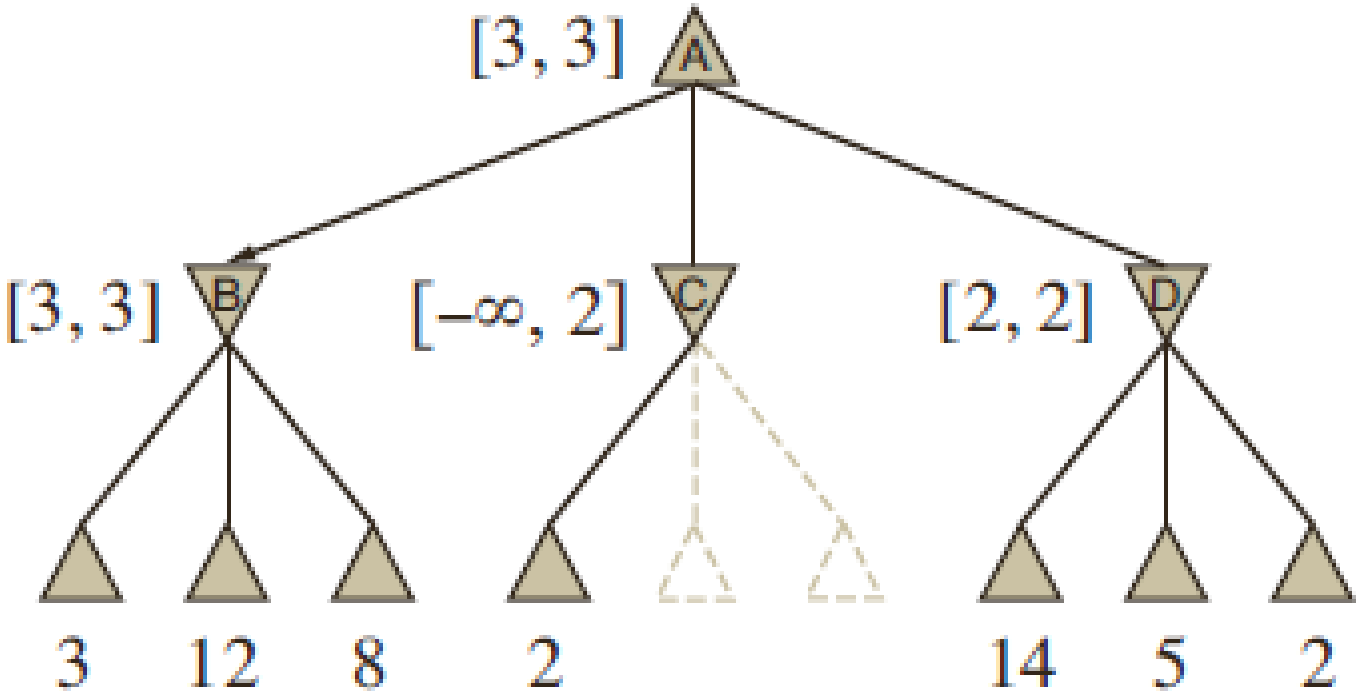
Alpha–Beta Pruning Example



Alpha–Beta Pruning Example

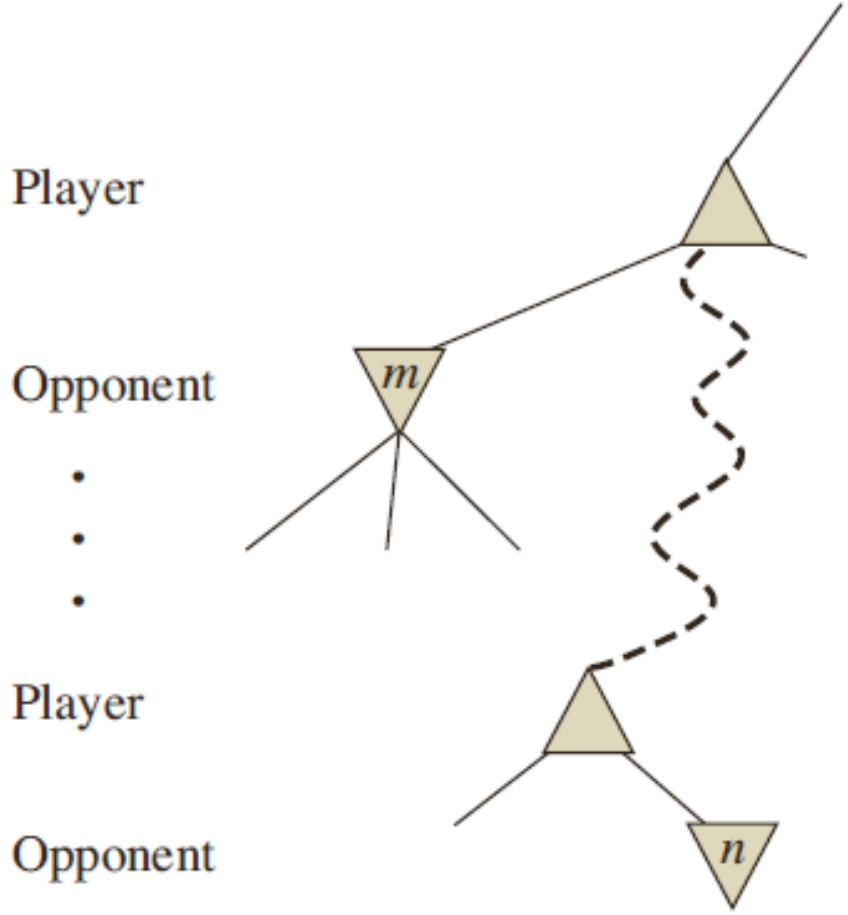


Alpha–Beta Pruning Example



The general case for alpha–beta pruning

- If m is better than n for Player, we will never get to n in play.



Alpha–Beta Search Algorithm

```
function ALPHA-BETA-SEARCH(state) returns an action
  v ← MAX-VALUE(state, −∞, +∞)
  return the action in ACTIONS(state) with value v
```

```
function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v
```

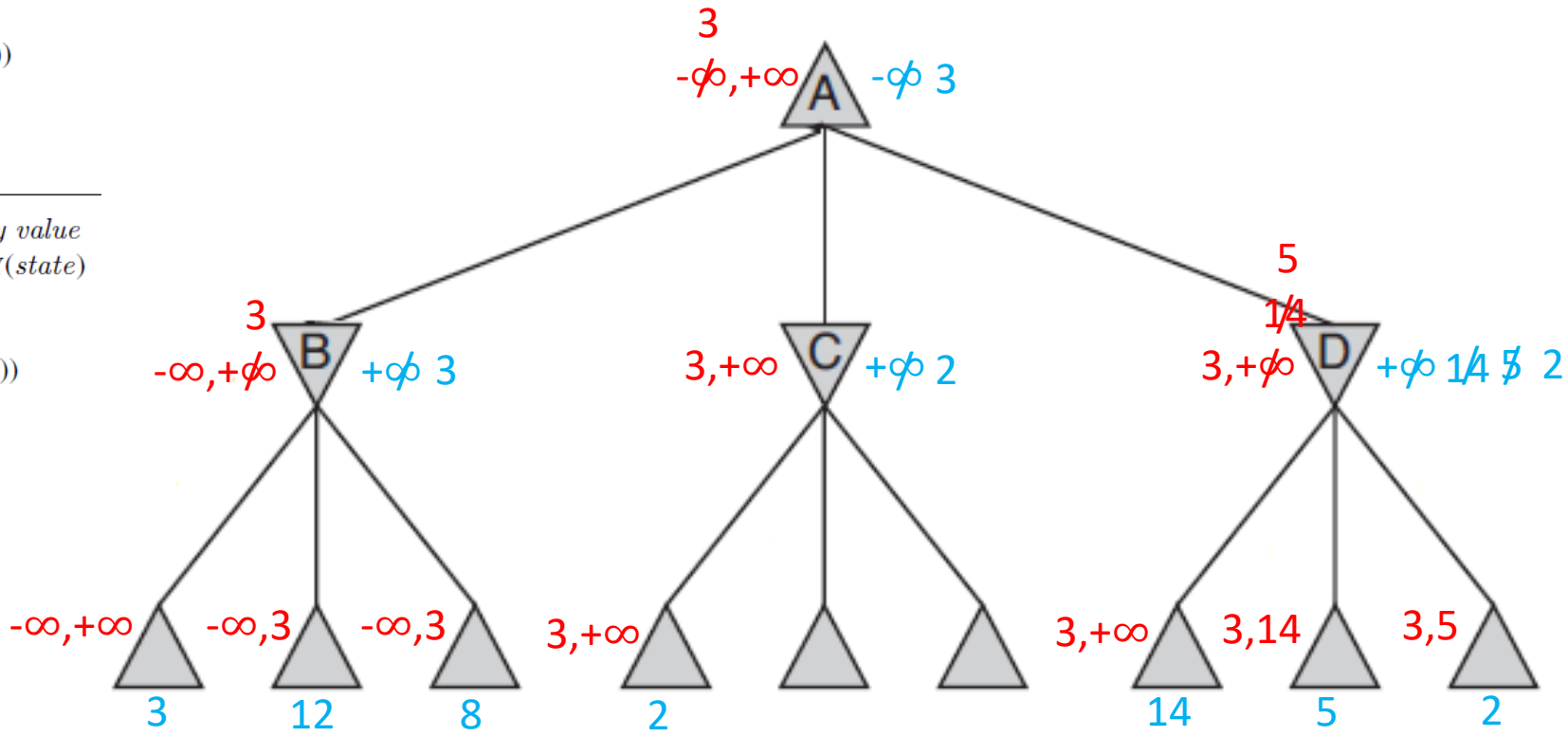
```
function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

Alpha–Beta Search Algorithm

```
function ALPHA-BETA-SEARCH(state) returns an action
  v ← MAX-VALUE(state, −∞, +∞)
  return the action in ACTIONS(state) with value v
```

```
function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v
```

```
function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```



Properties of Alpha–Beta Search Algorithm

- Pruning does not affect final result
- Good move ordering improves effectiveness of pruning
- With *perfect ordering*, **time complexity = O($b^{m/2}$)** ➔ doubles solvable depth
- Unfortunately, 35^{50} is still impossible! (for chess)

Imperfect Real-Time Decisions

- ***In order to make a move in a reasonable amount of time***, programs should ***cut off the search*** earlier and ***apply a heuristic evaluation function to states in the search***, effectively turning nonterminal nodes into terminal leaves.

- Replace the ***utility function*** by a **heuristic evaluation function EVAL**, which estimates the position's utility, and
- Replace the ***terminal test*** by a **cutoff test** that decides when to apply EVAL.
  - Depth-limit can be used in cutoff test.

- Heuristic minimax for state s and maximum depth d:

$$\text{H-MINIMAX}(s,d) = \begin{cases} \text{EVAL}(s) & \text{if CUTOFF-TEST}(s,d) \\ \max_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if PLAYER}(s) = \text{MIN.} \end{cases}$$

Example: Suppose we have 100 seconds, explore 10^4 nodes/second for chess

➔ 10^6 nodes per move $\approx 35^{8/2}$

➔ Alpha–Beta Search Algorithm reaches depth 8 ➔ pretty good chess program

Evaluation Functions

- An **evaluation function** returns ***an estimate of the expected utility of the game from a given position***.
- The *performance of a program* depends strongly on the ***quality of its evaluation function***.
  - An ***inaccurate evaluation function*** guides an agent toward ***losing positions***.

How exactly do we design good evaluation functions?

1. Evaluation function should order the terminal states same as the true utility function: win states must be better than draw states, draw states must be better than loss states.
2. The computation must not take too long! (The whole point is to search faster.)
3. For nonterminal states, evaluation function should be correlated with actual chances of winning.

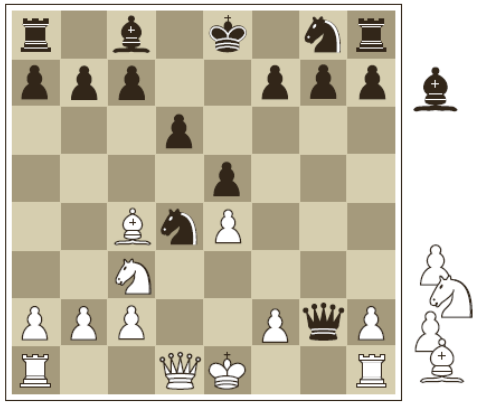
Evaluation Functions

- Most evaluation functions work by calculating **features of the state**.
- ***Evaluation function*** can be a **weighted linear function of features** expressed as

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^{n} w_i f_i(s)$$

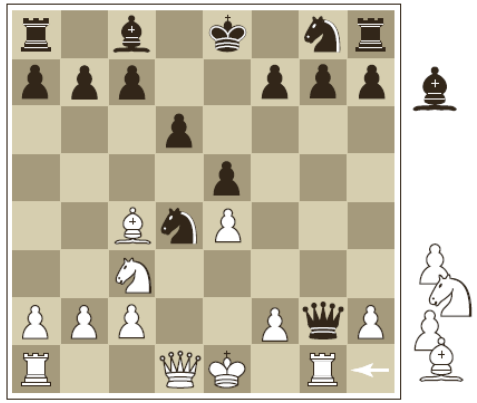
where w_i is weight and f_i is feature.

  - For chess, f_i could be numbers of each kind of piece on the board, and w_i could be values of pieces (1 for pawn, 3 for bishop, 3 for knight, 5 for rook, 9 for queen). This may NOT be a good evaluation function.



white to move

Black has advantage



white to move

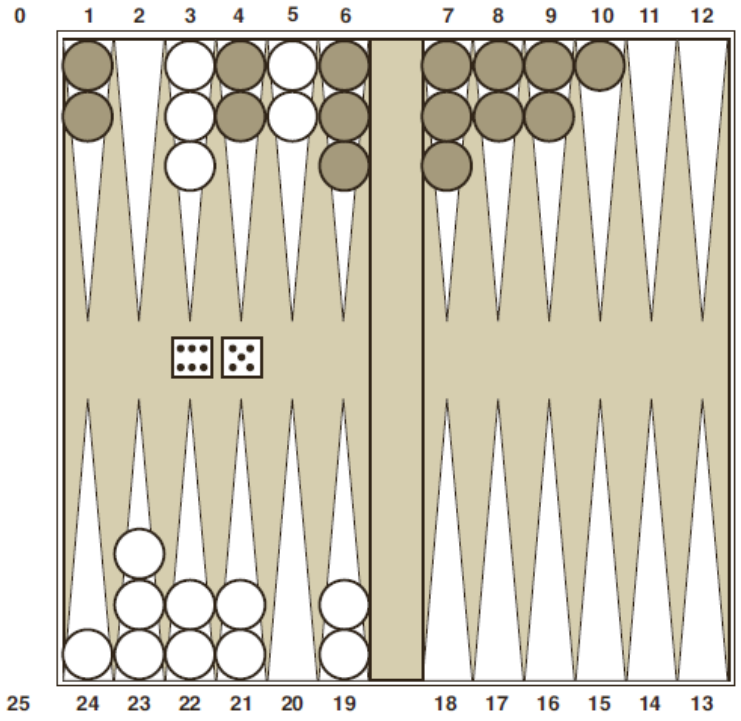
White has advantage

Stochastic (Nondeterministic) Games

- Many games mirror **unpredictability** by including a random element, such as the throwing of dice.

  ➔ They are called as **Stochastic (Nondeterministic) Games**

- **Backgammon** is a typical game that combines luck and skill.



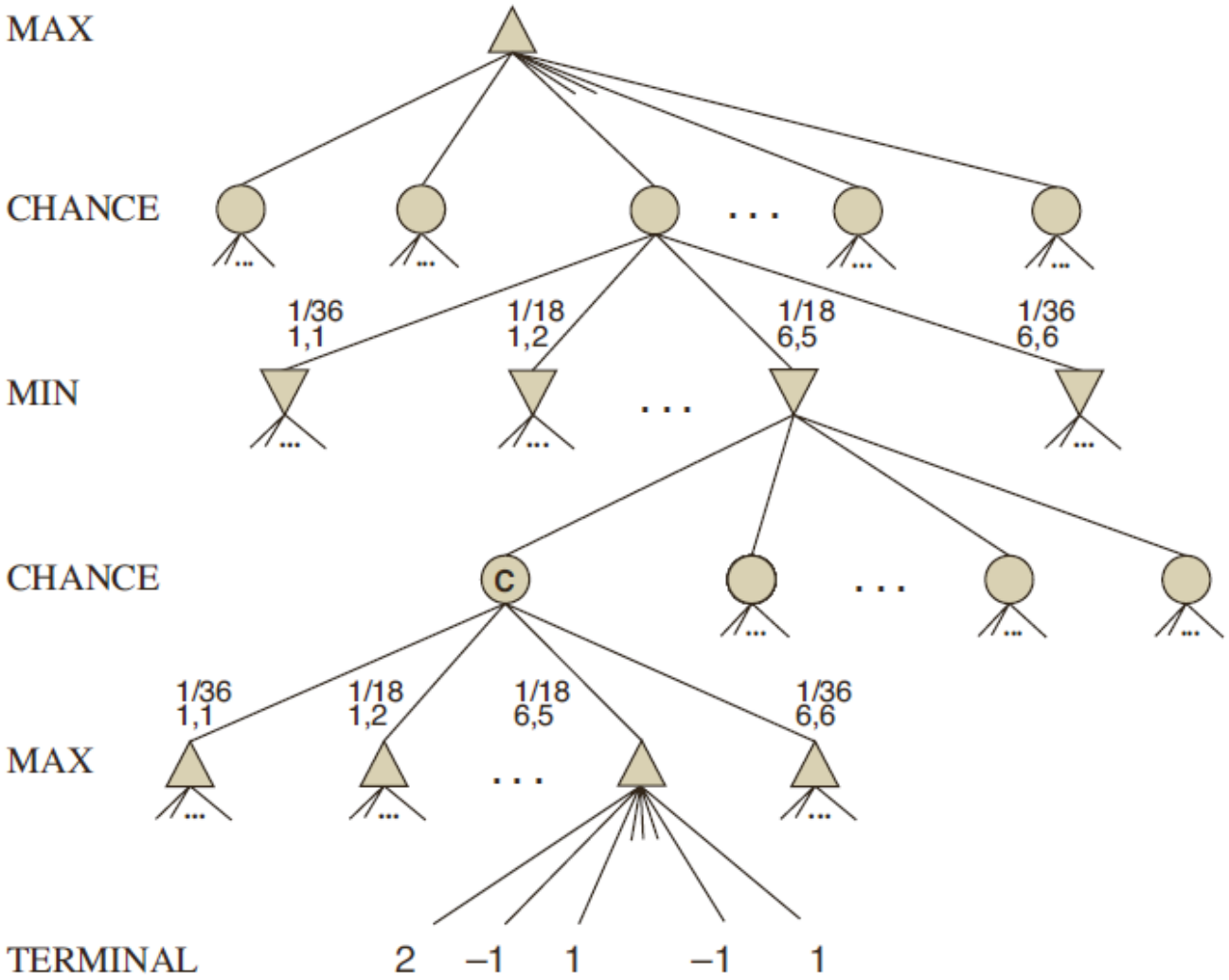
White has rolled a 6–5 and has four possible moves.

(5–10,5–11), (5–11,19–24), (5–10,10–16), and (5–11,11–16)

Stochastic (Nondeterministic) Games

- In backgammon, although White knows what his own legal moves are, but White does not know what Black is going to roll and thus does not know what Black's legal moves will be.
- That means White cannot construct a standard game tree of the sort we saw in chess and tic-tac-toe.
- A game tree in backgammon must include **chance nodes** in addition to MAX and MIN nodes.

Game Tree for a Backgammon Position



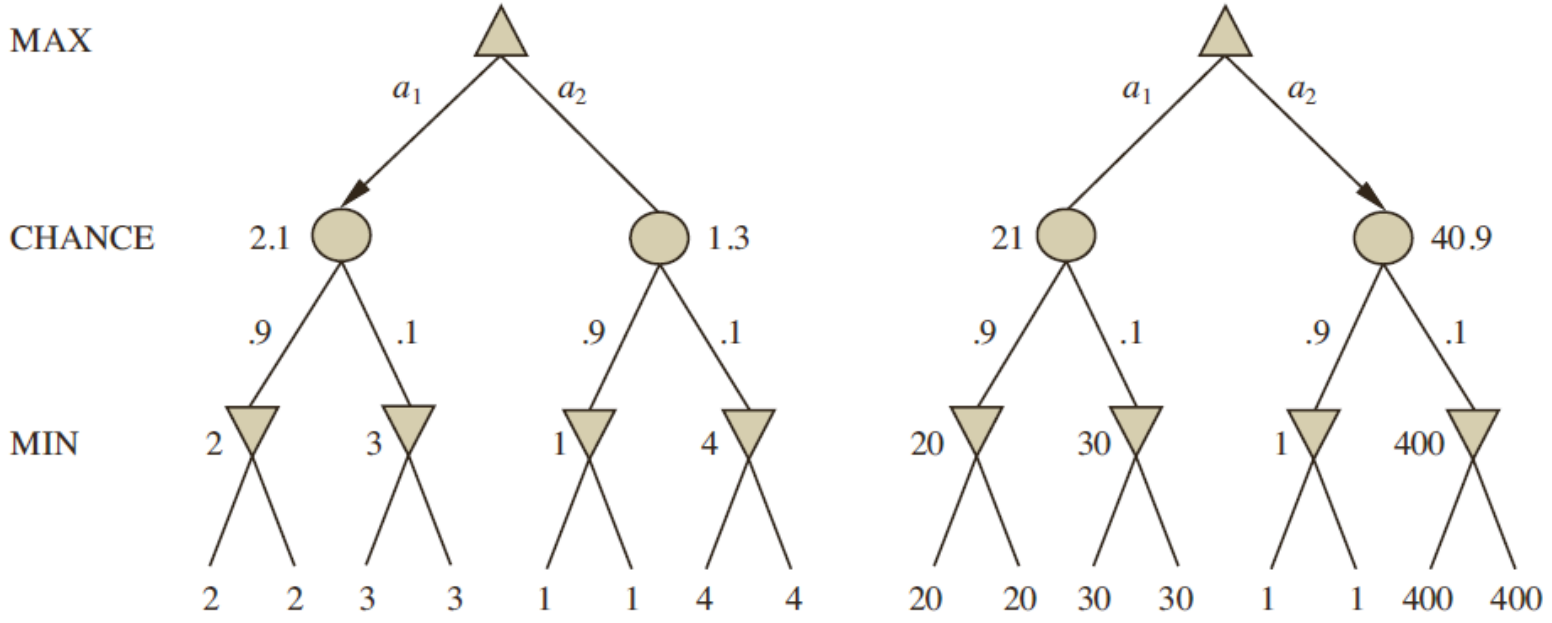
How to Make Correct Decisions?

- We still want to *pick the move that leads to the best position*.
- However, ***positions do not have definite minimax values***.
- Instead, we can only calculate the **expected value of a position**: the average over all possible outcomes of the chance nodes.
- We can generalize the minimax value for deterministic games to an **expecti-minimax value** for games with chance nodes.

$$\textsc{Expectiminimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } \textsc{Terminal-Test}(s) \\ \max_a \textsc{Expectiminimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{max} \\ \min_a \textsc{Expectiminimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{min} \\ \sum_r P(r)\textsc{Expectiminimax}(\textsc{Result}(s,r)) & \text{if } \textsc{Player}(s) = \textsc{chance} \end{cases}$$

Evaluation functions for games of chance

- Evaluation functions for backgammon should be just like evaluation functions for chess.
- Behavior is preserved only by positive linear transformation of EVAL
- **Hence EVAL should be proportional to the expected payoff**.



Games of Imperfect Information

- Card games, where opponent's initial cards are unknown
- Typically we can calculate a probability for each possible deal
- Seems just like having one big dice roll at the beginning of the game.
- Idea: compute the minimax value of each action in each deal, then choose the action with highest expected value over all deals
- Special case: if an action is optimal for all deals, it's optimal.

GIB, current best bridge program, approximates this idea by

1) generating 100 deals consistent with bidding information

2) picking the action that wins most tricks on average

State-of-the-Art Game Programs

Chess:

- IBM’s DEEP BLUE chess program defeated world champion Garry Kasparov
- Deep Blue ran on a parallel computer with 30 IBM RS/6000 processors
- used alpha–beta search
- searched up to 30 billion positions per move, reaching depth 14
- evaluation function had over 8000 features
- used a large endgame database of solved positions containing all positions with five pieces and many with six pieces

State-of-the-Art Game Programs

Checkers:

- CHINOOK, which runs on regular PCs and uses alpha–beta search.
- Chinook defeated the long-running human champion in a match in 1990.
- Used database of all 444 billion endgame positions with 8 pieces

Othello:

- In 1997, the LOGISTELLO program defeated the human world champion.
- Humans are no match for computers at Othello

Summary

- A game can be defined by
  - the **initial state** (how the board is set up),
  - the **legal actions** in each state,
  - the **result** of each action,
  - a **terminal test** (which says when the game is over), and
  - a **utility function** that applies to terminal states.
- In *two-player zero-sum games* with **perfect information**, the **minimax algorithm** can select optimal moves by a depth-first enumeration of the game tree.
- The **alpha–beta search algorithm** computes the same optimal move as **minimax**, but achieves much greater efficiency by eliminating subtrees that are provably irrelevant.
- Usually, it is not feasible to consider the whole game tree so we need to cut the search off at some point and apply a heuristic evaluation function that estimates the utility of a state.

Summary

- Many game programs ***precompute tables of best moves in the opening and endgame*** so that they can look up a move rather than search.
- Games of chance can be handled by an extension to the minimax algorithm that evaluates a **chance node** by taking the average utility of all its children, weighted by the probability of each child.
- Optimal play in games of **imperfect information**, such as bridge, requires reasoning about the current and future **belief states of each player**. A simple approximation can be obtained by averaging the value of an action over each possible configuration of missing information.
- Programs have bested even champion human players at games such as chess, checkers, and Othello. Humans retain the edge in several games of imperfect information, such as poker, bridge